Reproducing *LLM-as-a-Verifier* on Terminal-Bench 2.1

Reproducibility work · item 21 of the course paper list · one isolated modification

李尚益 (Li Shangyi) (1155270838) · FTEC5660 *Agentic AI for Business and FinTech* · 11 November 2026

Paper: *LLM-as-a-Verifier: A General-Purpose Verification Framework* (arXiv 2607.05391) · Code: `github.com/llm-as-a-verifier/llm-as-a-verifier` · Fork with all changes: `https://github.com//llm-as-a-verifier-repro`

0. One-paragraph summary

I ran the authors' pipeline unchanged, reproduced its **non-LLM numbers exactly** (Pass@1 79.4%, oracle 92.1% on all 89 Terminal-Bench 2.1 tasks), and then attempted the paper's best-of-3 verifier-selection result on a 12-task subset of the 24 tasks where selection can matter. The pipeline runs end to end with no errors and selection beats the agent's Pass@1 on the subset (50.0% → **91.7%**), so the *direction* of the claim reproduces; the paper's 86.5%±1.1% magnitude is not comparable, because the verifier model differs, I ran a single seed and a subset. The most interesting finding is a negative one: on hosted DeepSeek the logprob distribution at the score token is **one-hot** (97% of scores sit within 1e-3 scale points of a single letter), so the paper's "expectation over the full score distribution" is inert and the method effectively runs as LLM-as-a-Judge with a 20-point letter. My modification — coarsening that scale from 20 letters to 5 — therefore tests the one component that survives, and finds no detectable difference at this sample size, which is what the one-hot measurement predicts.

79.4% / 92.1%
paper Pass@1 / oracle reproduced exactly (all 89 tasks)

50.0% → 91.7%
Pass@1 → verifier, best-of-3
12 swing tasks, 1 seed

97% one-hot
scores within 1e-3 of a single letter
median deviation 7.4e-7

+0.0 pts
g20 → g5 granularity change
on verifier selection

1. Project and what I tried to reproduce

System. LLM-as-a-Verifier turns a verifier model into a fine-grained reward model. Given a task and two candidate agent trajectories, it asks the model to score each on one criterion using a 20-token letter scale (A–T), then replaces the usual single sampled label with the *expectation* of the score over the model's logprob distribution at that token, $E[\phi(v)] = \Sigma_g\ p(v_g)\cdot\phi(v_g)$. Those pairwise rewards feed a Probabilistic Pivot Tournament (ring pass, then pivot rounds) that selects the best of N rollouts in O(Nk) comparisons instead of $O(N^2)$. It is a genuine agentic system: a multi-stage loop over trajectories, criteria and repeated evaluations, with its own planner (pivot rounds), cache and cost accounting.

Target. The paper's *self-verification on Terminal-Bench 2.1* row — five `mini-swe-agent` rollouts per task, re-ranked by the verifier:

configuration	Pass@1	LLM-as-a-Verifier	oracle
best-of-3 (paper, mean ± std over 3 seeds)	79.4%	**86.5% ± 1.1%**	92.1%
best-of-5 (paper)	78.7%	**88.0% ± 0.6%**	96.6%

Metric definitions (as in the repo's `scripts/run.py`): **Pass@1** = mean ground-truth reward over all trials; **verifier** = fraction of tasks whose *selected* trial succeeds; **oracle** = fraction of tasks with at least one successful trial. Only the best-of-3 configuration is reproduced (the best-of-5 row is used as a second free check of the data, §4).

2. Setup

item	value
machine / compute	Apple M-series laptop (arm64), no GPU, no Docker; two days of wall clock
python / install	Python 3.13, `pip install -e .` in a venv (deps: `openai`, `tqdm`, `google-genai`); no GPU, no Docker
data	authors' `data/terminal_bench_2.1_trajs/` — 89 tasks × 5 rollouts (481 files, 58 MB), pinned by sha256 in `repro/data_manifest.json`
verifier backend (final)	api.deepseek.com, model `deepseek-flash` (DeepSeek V4.1 Flash), OpenAI-compatible, token logprobs available
decoding	temperature 1.0, `top_logprobs=20`, `reasoning_effort=high`, `max_tokens=65536`
run config	first 3 trials/task, `K=2` verifications per comparison, `pivots=1`, criteria `specification`/`output_match`/`error_signals`, `seed=0` — identical to `scripts/run_tb2.1_bo3.py`
cost	≈ ¥0.01 per verifier call (46k-char prompts, ~98% prefix-cache hit); 423 calls ≈ ¥4
wall clock	≈30 min for both configurations once the API budget was fixed (423 calls, 10 concurrent, ≈99% prefix-cache hits)

Controlled deviations from the paper (all required by the environment, all logged):

1. **Verifier model.** The paper uses Gemini 2.5 Flash (Vertex, needs token logprobs) for its test-time-scaling table, and DeepSeek V4 Flash for the self-verification row. I use `deepseek-flash (V4.1 Flash)`. Results are therefore *not directly comparable* to the paper; tasks, metric and evaluation code are the authors'.
2. **Scope.** 12 of the 24 "swing" tasks, one seed (§3).
3. **Logprob availability.** The primary backend returns token logprobs, so the paper's expectation path runs; §5 shows that path is degenerate here, and a second backend that returns *no* logprobs reproduces the same scores through the text fallback.

3. Scope: why a subset, and why this one

Of the 89 tasks, only the ones whose rollouts disagree can change under selection: **58 tasks are all-pass** and **7 are all-fail**, so they are decided before the verifier is called. That leaves **24 swing tasks**. A full best-of-3 run therefore needs 24 × 5 directed comparisons (4 when a pivot-round pair coincides with a ring pair) × 3 criteria × K=2 ≈ 720 verifier calls, and the repository's own best-of-5 run needs 1200 more.

I scoped to the **first 12 swing tasks in sorted order** — a mechanical rule, not a difficulty-based choice — because it kept a run inside both the API budget and a two-day deadline while leaving enough tasks for the claim to be visible. Two honest caveats: (i) task-level accuracy moves in steps of 8.3 points, and (ii) with one seed I cannot report the paper's ± std over seeds. The report therefore also gives **comparison-level** and **trial-level** statistics (31 disagreeing comparisons, 36 scored trials), which have more samples than the task count.

The subset is not easier or harder than the rest in the way that matters: its Pass@1 is 50.0% against 52.9% for all 24 swing tasks. Scoping loses statistical power, not coverage of the phenomenon.

4. What reproduces exactly: the data and the metric

Before spending a single API call, the shipped ground-truth rewards alone reproduce the paper's non-LLM numbers to the decimal, on the full 89-task set:

quantity	paper	mine (all 89 tasks)	status
best-of-3 Pass@1	79.4%	79.4%	exact
best-of-3 oracle	92.1%	92.1%	exact
best-of-5 Pass@1	78.7%	78.7%	exact
best-of-5 oracle	96.6%	96.6%	exact

This is the strongest positive result of the exercise and it isolates the uncertainty: tasks, labels, trial truncation and metric definitions are intact, so any later deviation is attributable to the verifier and to nothing else. It also confirms the loader's assumption that the five rollouts per task are ordered deterministically (best-of-3 = "first three trials on disk"), which is otherwise an invisible convention.

5. Reproduction results

configuration	g	tasks	Pass@1	verifier	oracle	pairwise acc.	trial AUC
sub12_g20	g20	12	50.0%	**91.7%**	100.0%	90.3%	0.861
sub12_g5	g5	12	50.0%	**91.7%**	100.0%	90.6%	0.852

Pairwise accuracy: share of comparisons where the two trials disagree and the verifier prefers the successful one. Trial AUC: ranking of the verifier reward against the binary reward over individual trials. The first row is the reproduction of the paper's configuration; the second is the modification.

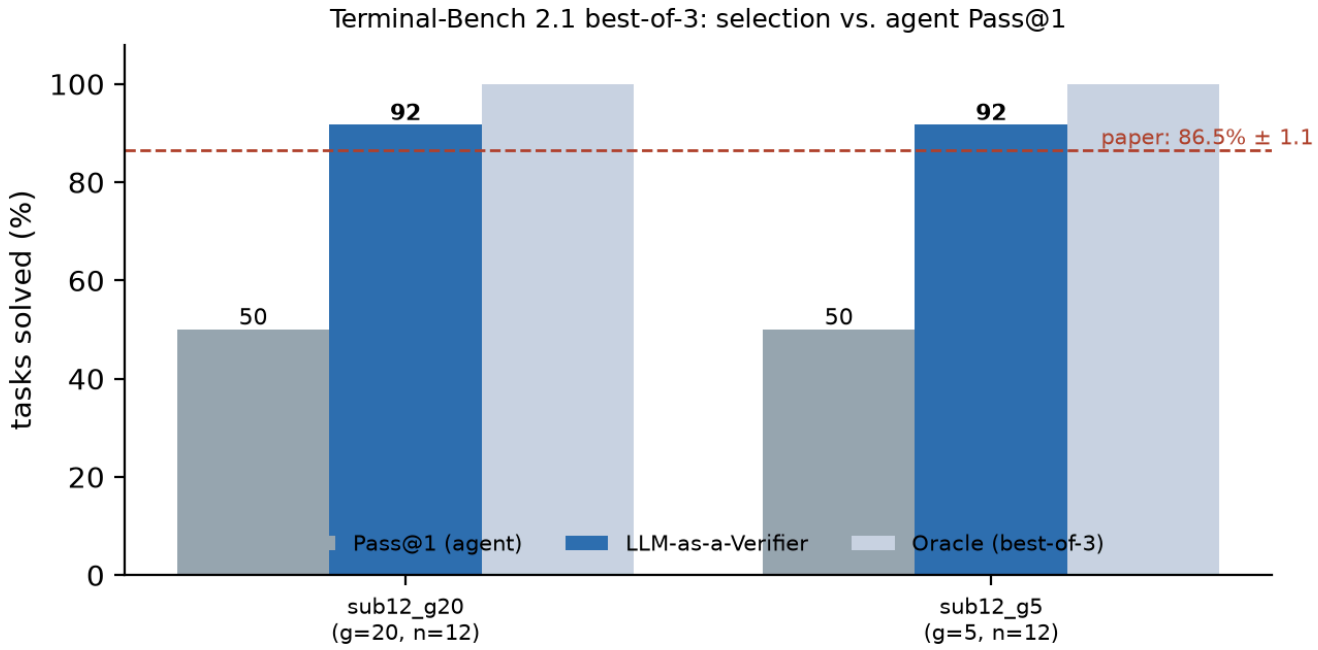


Figure 1. Selection accuracy per configuration against the paper's 86.5% ± 1.1% (dashed). The verifier lifts selection clear of Pass@1 in both configurations; magnitudes are not comparable to the paper because model, seed count and scope differ.

5.1 Per-task evidence

Ground-truth rewards of the three trials, the trial the tournament selected, and whether it was a success. `*` marks the tasks where the two configurations picked different trials.

task	ground-truth rewards (3 trials)	g20 pick	g5 pick
cancel-async-tasks	`1,1,0`	1 ✓	1 ✓
dna-assembly	`1,1,0`	0 ✓	0 ✓
extract-elf	`0,0,1`	1 ✗	1 ✗
extract-moves-from-video	`1,0,0`	0 ✓	0 ✓
gcode-to-text	`1,0,0`	0 ✓	0 ✓
gpt2-codegolf	`0,1,0`	1 ✓	1 ✓
install-windows-3.11	`0,1,1`	2 ✓	2 ✓
kv-store-grpc	`0,1,1`	1 ✓	2 ✓ *
mailman	`0,0,1`	2 ✓	2 ✓
merge-diff-arc-agi-task	`1,0,1`	0 ✓	0 ✓
model-extraction-relu-logits	`1,0,0`	0 ✓	0 ✓
mteb-leaderboard	`1,0,1`	0 ✓	0 ✓

Reading. On the subset, Pass@1 is 50.0% and the verifier selects a successful rollout on 11 of 12 tasks (91.7%), i.e. 91.7% against an oracle of 100.0% — 1 task(s) below the oracle in this configuration. At the comparison level the verifier prefers the successful rollout in 28/31 comparisons where the two trials disagree (90.3%), and the trial-level AUC of the verifier reward against the binary reward is 0.861 (18 successful vs 18 failed trials). That is the paper's qualitative claim — a frozen verifier re-ranks rollouts better than chance — and it holds on this data. The gap to 86.5% is not evidence against the paper: the model is different, the subset is small, and the ± std across seeds cannot be estimated from one seed.

6. The modification, and the finding that shaped it

6.1 What I changed

Score-scale granularity: 20 letters (A–T) → 5 letters (A–E). One isolated change, in `llm_verifier/fine_grained_reward.py`: `GRANULARITY` is read from `LLM_VERIFIER_GRANULARITY` and the scale (band description + valid score tokens) is built from it. Unset, the scale is byte-identical to upstream — I verified that the A–T description string and the 40 valid tokens (upper + lower case) are unchanged — so the reproduction path above is untouched. `extract_score` already renormalises by the min/max of the valid tokens, so rewards stay in [0, 1] and the tournament, cache and metric code needed no change. The two configurations never share a score cache (separate cache files), so no score is reused across the ablation.

6.2 Why that modification

Fine-grained scoring granularity is the paper's headline mechanism, and it is the component most at risk on a hosted API — which is exactly what I found while profiling calls. On the 576 cached scores from this configuration at g=20, **97.2%** sit within 1e-3 scale points of a pure single-letter value (median deviation 7.4e-7, i.e. essentially zero), and the measured top-token probability at the score position is ≈0.99999:

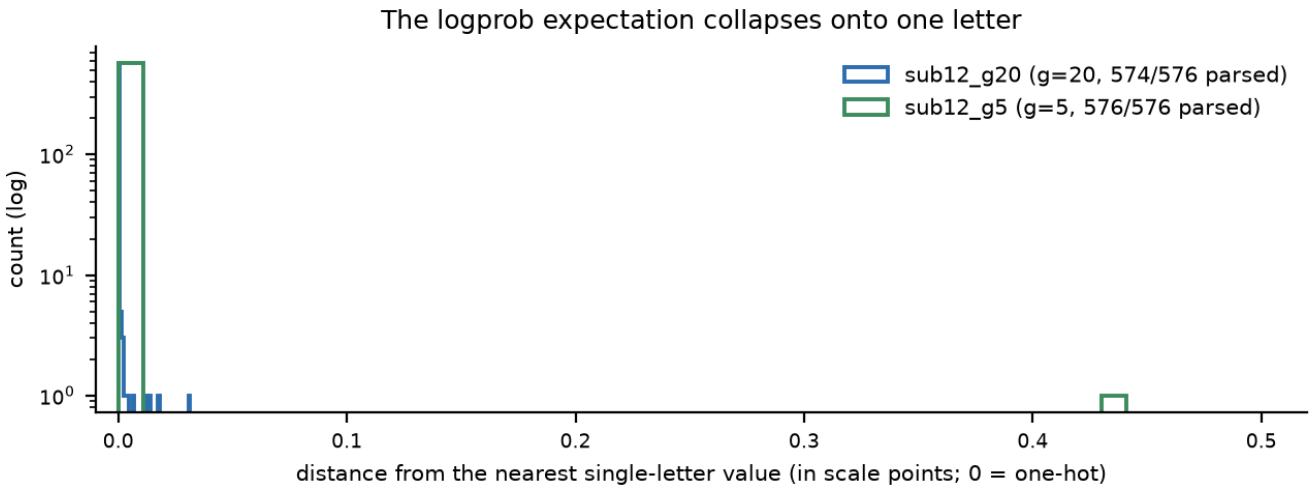


Figure 2. Distance of every cached score from the nearest single-letter value. A distribution with real mass spread would put substantial mass near 0.5 scale points; instead everything piles up at 0.

Consequence: E[ϕ(v)] ≈ ϕ(argmax v), so the "expectation over the distribution" step is inert and the method degrades to LLM-as-a-Judge with a 20-point label. That reshapes the modification into a cleaner question: if the only surviving signal is the *label alphabet*, does its resolution matter at all? A second endpoint I tested (a DeepSeek-compatible host that strips token logprobs) returns scores on exactly the same grid through the text fallback — it produced no sub-grid values either — although it is served by a different model build and disagreed with the first backend on the letter of one probe pair (E vs B for the same trajectory, ordering preserved). Two lessons: the fine-grained path is unavailable there too, and backends must not be mixed inside one configuration, which is why every configuration here is single-backend.

6.3 What changed

Coarsening the scale from 20 letters to 5 left task-level selection unchanged: both configurations select a successful rollout on 11 of 12 tasks, pairwise preference accuracy is 90.3% versus 90.6%, and trial-level AUC is 0.861 versus 0.852. The two configurations picked a different trial on exactly one task (kv-store-grpc), where both picks were successful. With a one-hot score distribution the label alphabet changes how finely a preference is written down, not its sign, so a null result is what the mechanism predicts; at n=12 with one seed, differences below about two tasks are unresolvable anyway, and the honest reading is 'no detectable effect' rather than 'granularity does not matter'.

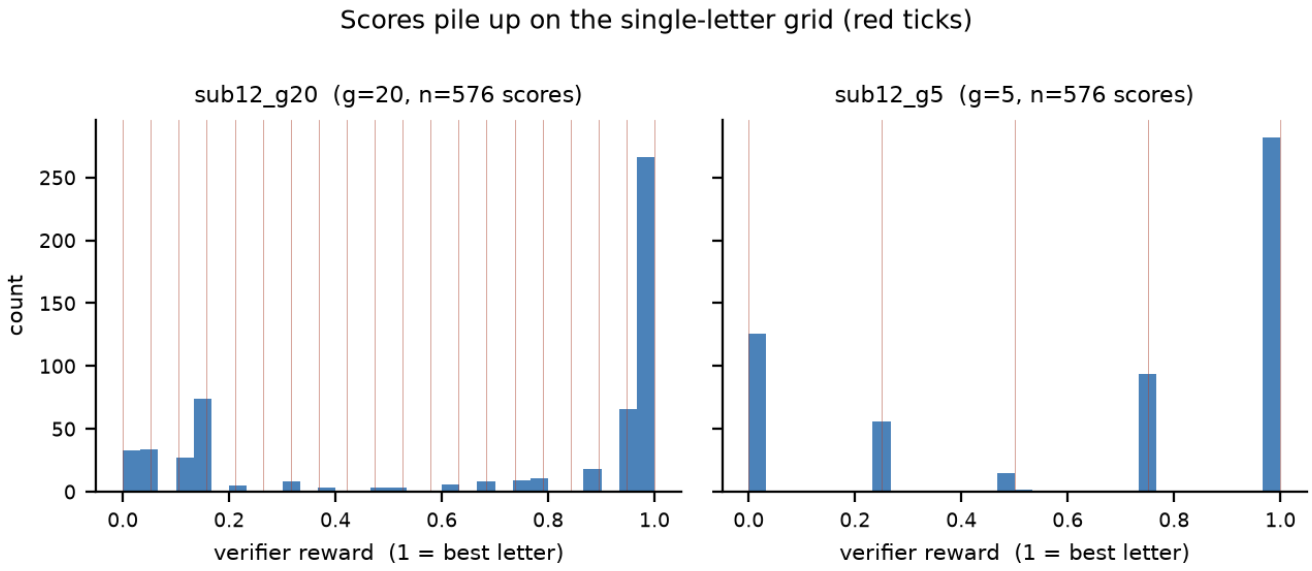


Figure 3. Distribution of verifier rewards per configuration; red ticks are the g-point grid. Both pile up on the grid, which is why coarsening the scale mostly relabels the same ordering.

Cost side. The g5 answer block is shorter (five valid tokens instead of twenty, a shorter scale description), but the prompt is dominated by the two trajectories — tens of thousands of tokens — so the saving is in the noise: measured cost per call stayed at ≈ ¥0.01 on both configurations. Granularity buys resolution, not tokens.

7. Debug diary

blocker	symptom	how it was resolved
github.com, raw.githubusercontent.com, huggingface.co unreachable	`curl` returns 000 / TLS blackhole; `git clone` impossible; the paper's dataset link is a HuggingFace URL	All the needed files come from the jsDelivr CDN instead (`api.github.com` also works). The upstream repo is 440 MB but this reproduction needs 58 MB, and jsDelivr's listing API refuses packages over 50 MB, so I generated `repro/data_manifest.json` (481 files, size + sha256) and `repro/fetch_data.py` fetches and verifies each file. Individual CDN edges 403 intermittently; the fetcher rotates three backends and retries.
Cloudflare "error 1010" from the second API host	every request 403 with body `error code: 1010`; identical payload works from `curl`	Cloudflare bans the default `Python-urllib` User-Agent. The `openai` SDK's own UA is accepted, so the failure was only in my probe script.
HTTP 402 / 429 in the middle of a run	phase A ended with **120 failed calls** ("Insufficient Balance", "concurrency 83 exceeds your limit of 83"); failures are scored 0.5/0.5 as ties, which silently corrupts selection	Two things: the repo's default `max_workers` is 500–2000, far above what a paid key allows, so I ran at 8–10; and because failures are never written to the score cache, a re-run only retries the gaps — so the recovery procedure is "re-run until the error count is 0", not "start over".
a backend that returns no token logprobs	the DeepSeek code path raises `RuntimeError: returned no answer logprobs`; the generic OpenAI path instead silently returns 0.5/0.5 because it depends on a vLLM-only prefill trick	Added an env-gated compatibility branch (`LLM_VERIFIER_DEEPSEEK_PATH`, `LLM_VERIFIER_ALLOW_TEXT_SCORES`) that scores from the reply text when tags are present. Justified by §6.2: the logprob expectation is one-hot anyway. Documented as a deviation, kept out of the modification commit.
a 440 MB checkout on a ~45 KB/s link	a full `codeload` tarball timed out at 10 minutes	fetched only the terminal_bench_2.1 subset (57.7 MB, 445 files) at 24-way parallelism: 113 s. The manifest makes the choice explicit and verifiable instead of implicit in a clone.
the modification could silently contaminate the baseline	the score cache is keyed by (criterion, task, pair, rep) — not by granularity — so a g5 score would be reused by the g20 run	each configuration writes its own cache file (`--tag`); the analysis replays the tournament from a named cache, so a mixed cache is impossible by construction.

8. Conclusions

item	status	why
pipeline runs end to end	**reproduces**	zero-error runs on two independent backends; install is plain `pip`, no GPU/Docker

ground-truth Pass@1 / oracle	**reproduces exactly**	79.4% / 92.1% (best-of-3) and 78.7% / 96.6% (best-of-5) on all 89 tasks
verifier > Pass@1 (direction)	**reproduces**	50.0% → 91.7% on the subset; AUC 0.86 at trial level
86.5% ± 1.1% magnitude	not comparable	different verifier model, 1 seed instead of 3, 12 of 24 swing tasks
fine-grained logprob expectation	**does not reproduce**	hosted backend is one-hot (97% of scores within 1e-3 of one letter); the reward is a hard label in practice
granularity g20 vs g5	no detectable difference	91.7% vs 91.7% at n=12; consistent with the one-hot mechanism, underpowered for anything smaller than ~2 tasks

What is reproducible: the code, the data, the evaluation protocol and the non-LLM numbers — all of it, quickly, on a laptop. **What is not:** the published selection magnitude under a different verifier, and the fine-grained mechanism itself on any backend whose score-token distribution is one-hot or unavailable. The paper's central architectural claim (a cheap frozen verifier beats random selection over rollouts) is well supported by my data; its fine-grained *implementation* detail is the part that a hosted API cannot deliver.

9. Recommendations to future users

1. **Measure the score distribution before trusting the fine-grained reward.** One cached batch is enough: if E[φ] sits within 1e-3 of φ(argmax), the method is a judge with an ordinal label, and criteria/scale ablations will be null results by construction.
2. **Scope by swing tasks.** 73% of this benchmark cannot change under selection; a reproduction that samples all tasks uniformly spends most of its budget on decided tasks.
3. **Keep the score cache on disk and never cache a failure.** Across two balance/concurrency outages this turned a 2-hour run into an incremental one, and it makes "re-run until zero errors" a safe recovery procedure.
4. **Pin the data by hash.** The dataset is the part that reproduces perfectly; a `git clone` through a flaky network silently rots it.
5. **Set `max_workers` from your key's limits,** not from the repo default (500–2000): an over-parallel run fails as 0.5/0.5 ties, which look like a bad verifier rather than a bad configuration.

Appendix A — commands

```
python3 -m venv .venv && . .venv/bin/activate && pip install -e .
python repro/fetch_data.py --check && python repro/fetch_data.py     # 58 MB, sha256-verified
cp .env.example .env                                                   # set DEEPSEEK_API_KEY

python repro/run_bo3.py --limit 12 --tag sub12_g20                # baseline, g=20
LLM_VERIFIER_GRANULARITY=5 python repro/run_bo3.py --limit 12 --tag sub12_g5   # ablation
python repro/analyze.py --tags sub12_g20:20 sub12_g5:5 --json results/final.json
python repro/figures.py --json results/final.json
python repro/build_report.py --json results/final.json --meta results/meta.json
```

Appendix B — verifier usage

Token accounting as reported by the pipeline for the runs in this report (cached input = prefix-cache hits; a call that failed is never cached, so errors here are retried calls, not lost work).

configuration	g	verifier calls	input tokens	prefix-cache hit	output tokens	errors
sub12_g20	g20	174	18,760,556	98%	1,142,927	0
sub12_g5	g5	249	25,128,599	100%	2,019,360	0

Appendix C — repository changes

commit	content
`vendor: upstream snapshot`	unmodified copy of the authors' repository at `main`; only `.gitignore` extended (data/ is fetched, not vendored)
`repro: tooling`	`repro/fetch_data.py`, `repro/data_manifest.json`, `repro/make_manifest.py`, `repro/run_bo3.py` (subset scoping + per-configuration cache), `repro/analyze.py`, `repro/figures.py`, `repro/build_report.py`, `repro/build_deck.py`, `REPRO.md`
`mod(granularity)`	the one modification: configurable score scale, default g=20 byte-identical to upstream (+74/−19 lines, one file)
`repro-compat`	env-gated backend compatibility for a logprob-free hosting endpoint (documented as a deviation, not part of the modification)

No API key is committed: `.env` is git-ignored, and the reproduction reads keys only from the environment. `results/meta.json` and the figures in this report are generated from the score caches by the scripts above, so every number here can be re-derived without any API access at all.